

Project 1 Design and Documentation: Shivam Sinay Kharangate

Forest Explorer

Source code: <https://github.com/ShivamKGate/ComputerGraphics2026>

Live application: <https://shivamkgate.github.io/ComputerGraphics2026/proj1/>

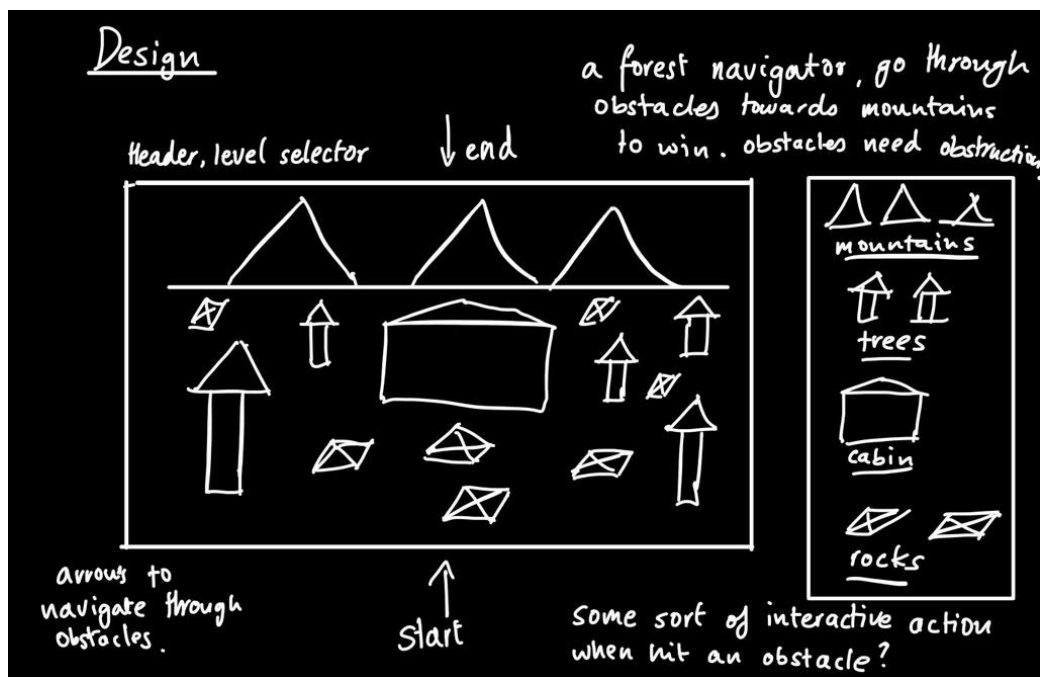
Code file: <https://github.com/ShivamKGate/ComputerGraphics2026/blob/main/proj1/index.html>

Demo video (Download required):

<https://github.com/ShivamKGate/ComputerGraphics2026/blob/main/proj1/Proj1-CG-sinayksp-demo.mp4>

Design

The idea for this project was a simple first-person view game of navigating through a forest. The camera isn't rotational, it just goes forward, back, left or right through the arrow keys or WASD for ease of movement. If you navigate through the trees, rocks, and the cabin in the middle to reach between the mountains towards the end, you win the game. The design proposed for this project is shown below in the image.



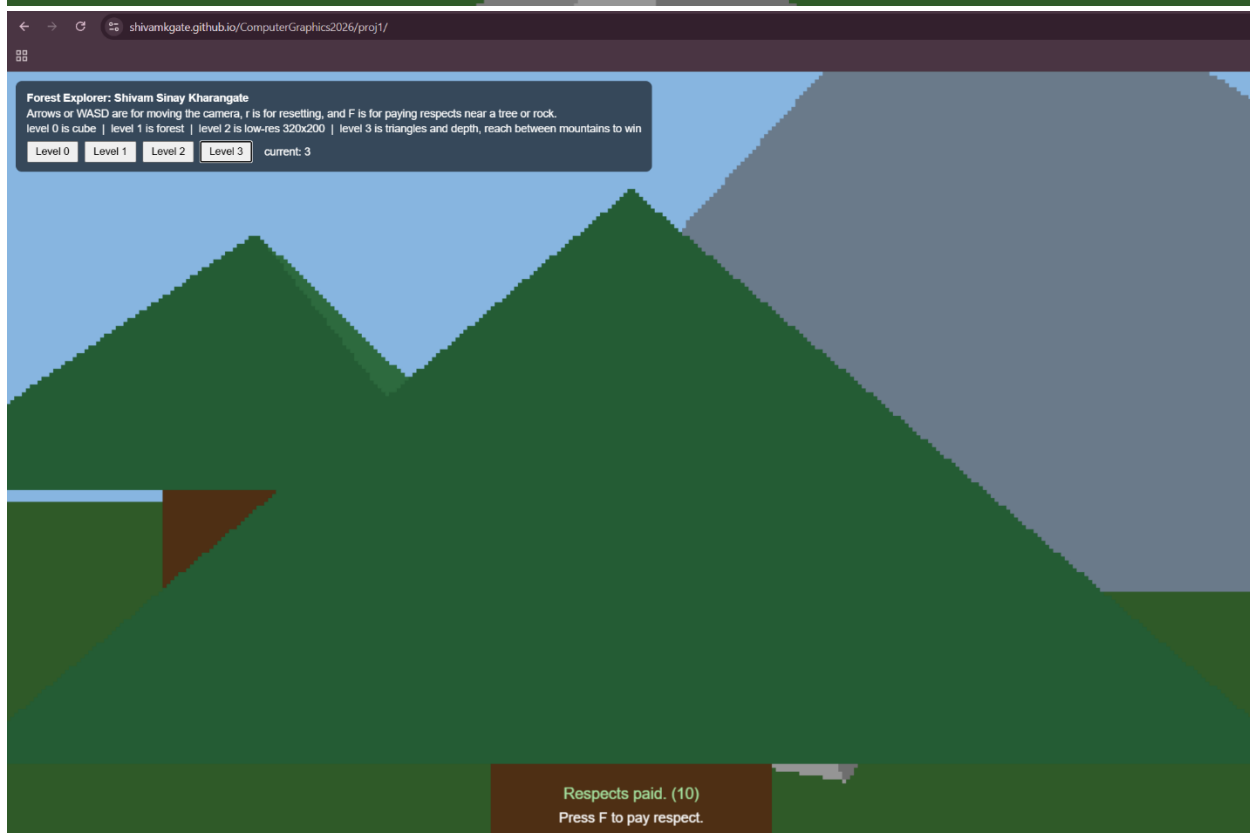
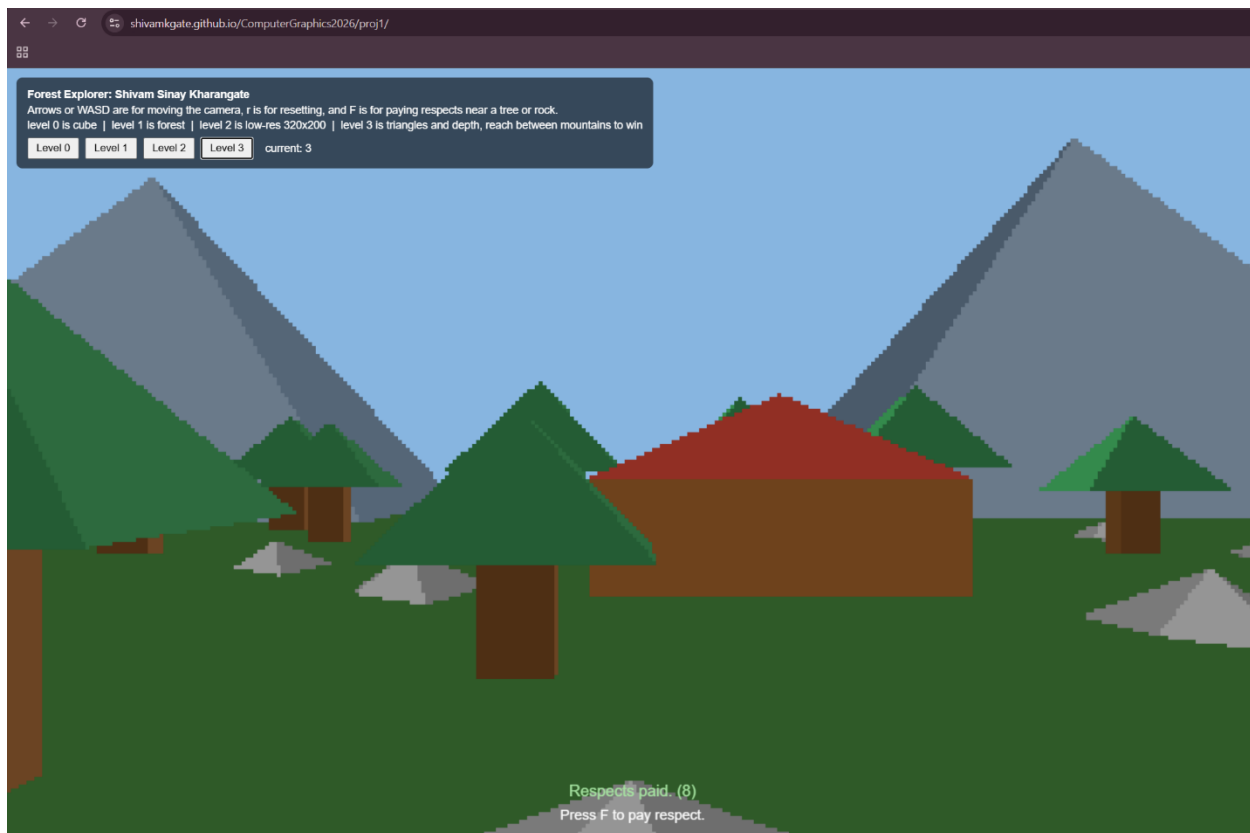
The Controls

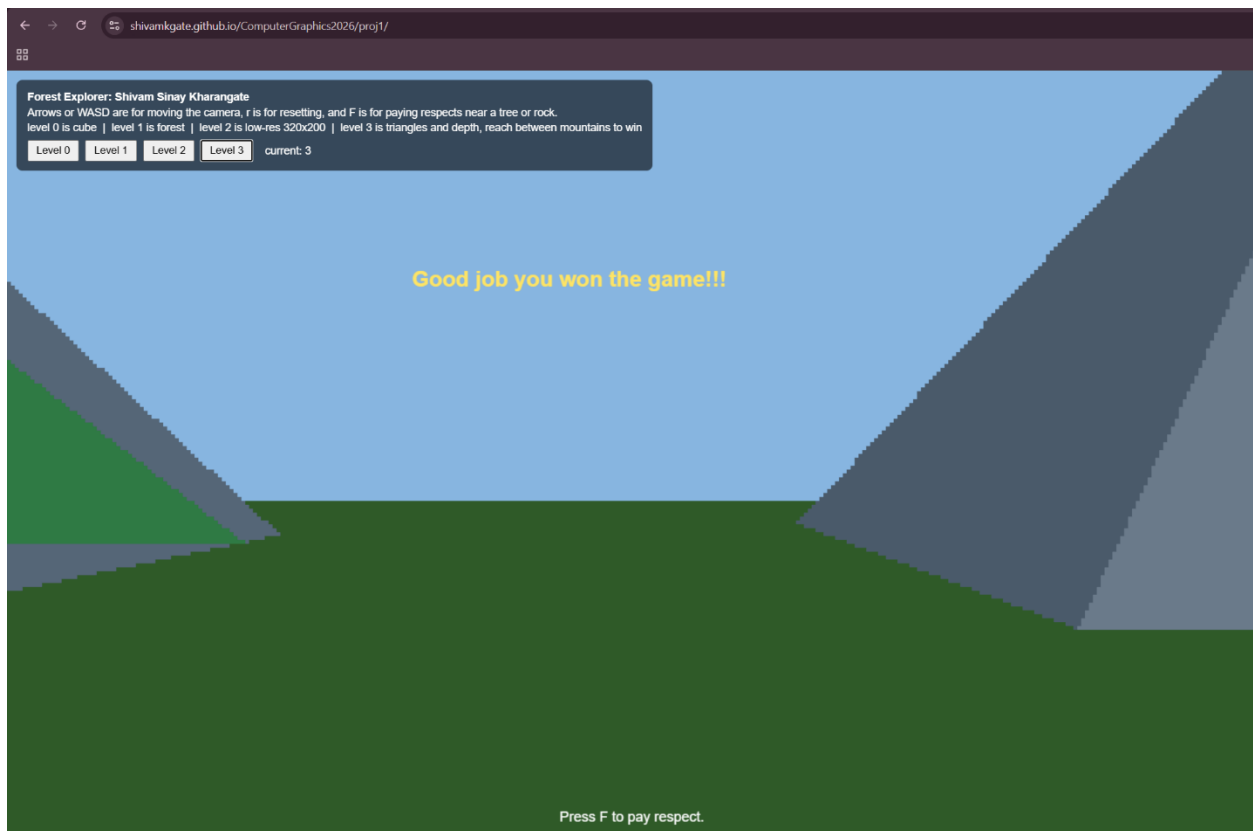
The page has a header with the title, my name, and how to use the controls, with a canvas. The buttons and number keys switch the levels. The same forest is used in levels 1 to 3. Only the implementation of the method as described in the project specifications changes.

Control	What it does
Arrow keys or WASD	Moves the camera, only x and z
r	Resets the current level to start state
F	If you're near a tree/rock: "Press F to pay respect."
Level 0	Cube only with controls, the starter scene
Level 1	Forest wireframe, normal canvas lines
Level 2	Same forest, custom lines on a 320x200 grid
Level 3	Same forest, filled with triangles and depth

There is custom logic implemented through which you cannot walk through the trees or rocks scattered throughout the forest, and the cabin that is centered in the middle of the forest. If a step would land inside one, then that would cancel the move. If you navigate through the objects and reach between the mountains, a floating win message appears.





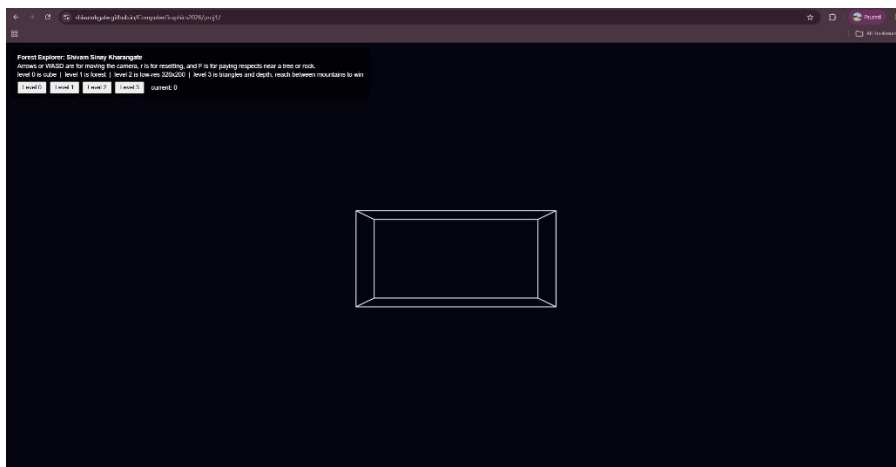


Implementation

Everything is built under one index.html file. The link to this file is attached

Models

Each object is defined in the object space, centered around the origin as vertices and edges. This is the same idea as the cube activity we did, the corners come first, then which corners connect. The objects are a tree (trunk and canopy), rock, cabin, and finally the mountains. The cube is present for level 0.



Instances

A model is drawn more than once by storing a position and then scaling on each copy. The trees and rocks are different sizes, which helps in distinguishing them. The project only translates and scales from the 2D transformations, having the object space to world space. In the code, each vertex goes through: $\text{world} = \text{vertex} * \text{scale} + \text{position}$

Pinhole Camera

The camera is a point. For each world vertex:

1. Subtract the camera: $\text{vertex} - \text{camera}$.
2. If the camera-space (z) is too small, then we skip it, as the point is behind or too close.
3. Project with the formulas $u = x/z$ and $v = y/z$.
4. Finally, scale and center that onto the canvas, and then flip v because the canvas y grows downwards.

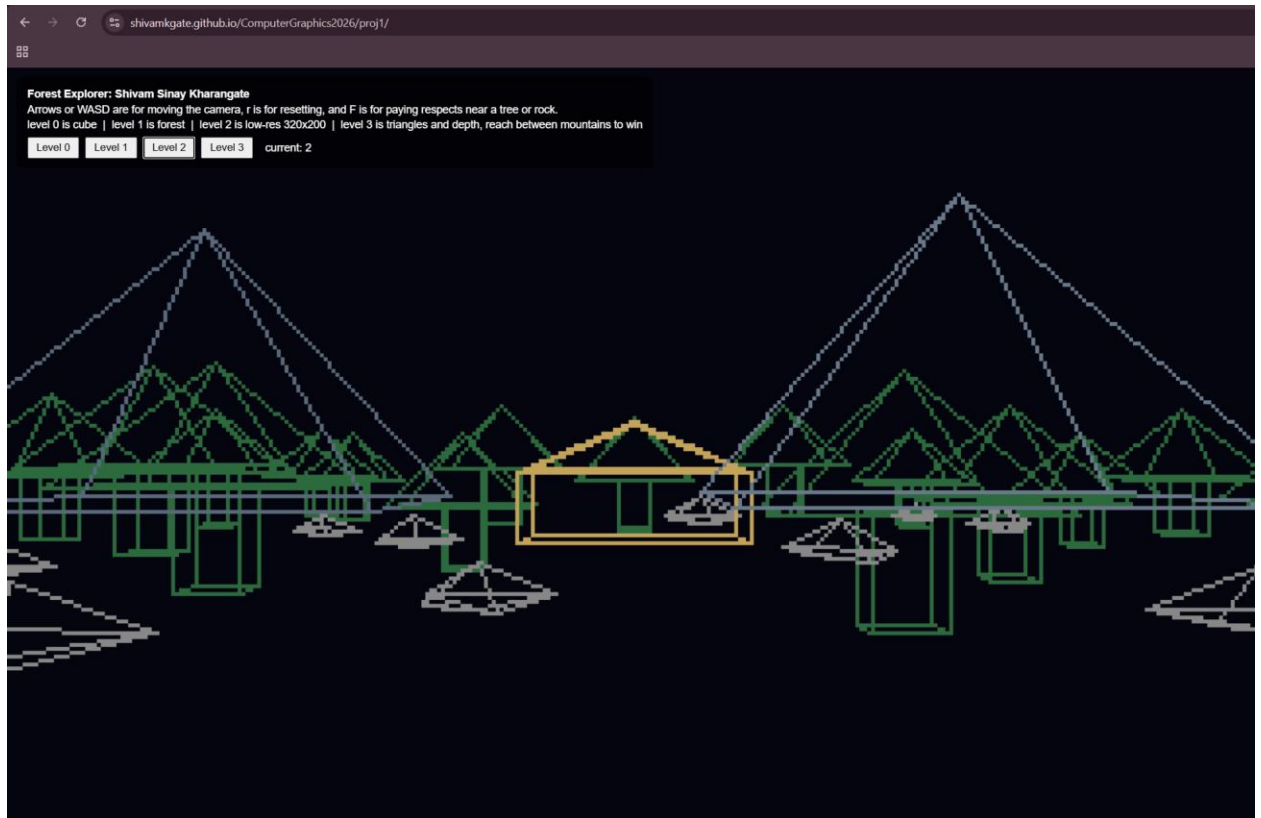
The left/right movements change the camera x. The forward/back movements change the camera z. The camera does not rotate, it stays stationarily forward. The collision logic is a circle around the trees, rocks and the cabin with the radiuses, so the camera cannot pass through them when centered in front of them.

Level 1: Canvas Lines

Level 1 uses the built-in canvas line (beginPath, moveTo, lineTo, stroke) on the canvas. Each of the objects types have a color specified for them, so they're drawn in different colors to distinguish them.

Level 2: Low-Res 320x200 Line Rasterization

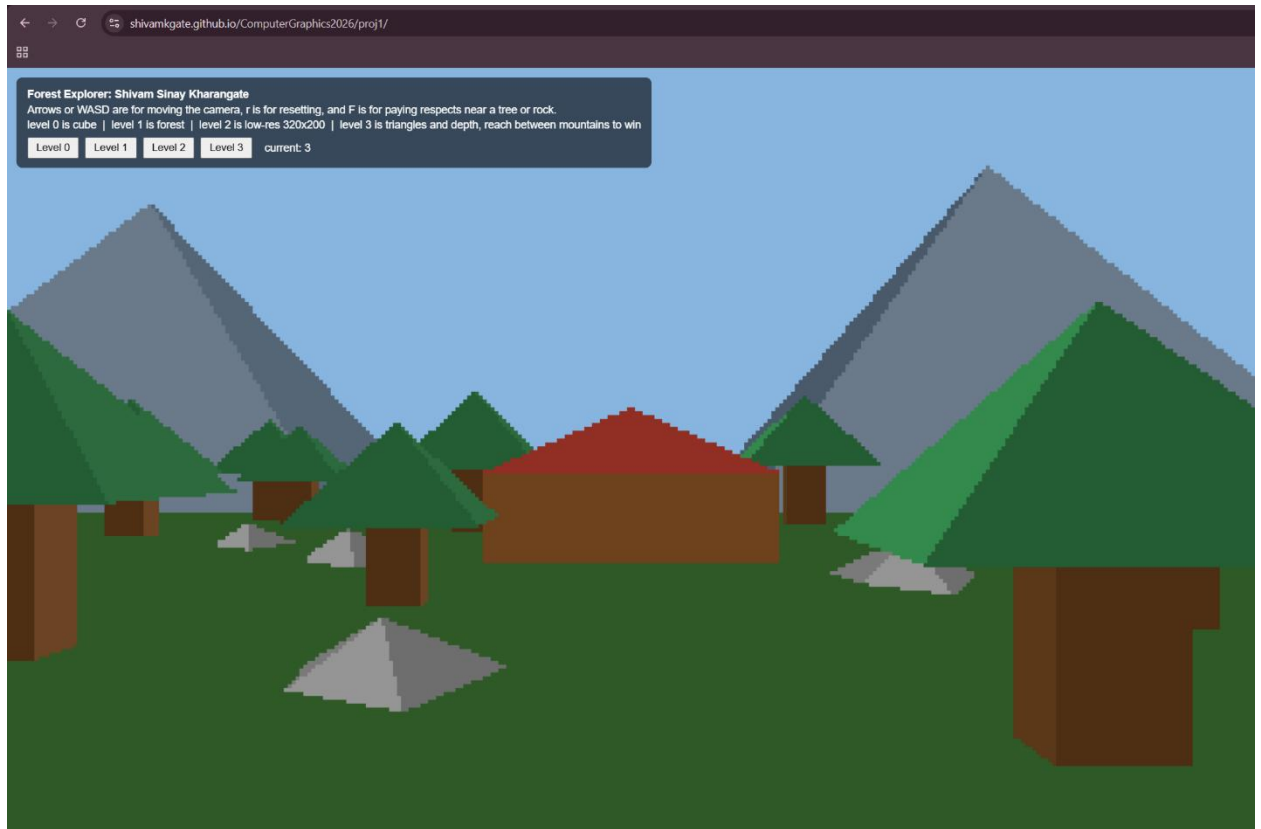
Level 2 uses the pixel-grid logic, which is a 320x200 array, first filled with the background color. Then, my own line function which is incremental so that it works for slopes besides 0 and 1 writes colors into that same array. After every edge is done, each cell is drawn as a 5x5 rectangle. This is rasterization, where a continuous line becomes a set of pixels on a grid instead of a canvas stroke.



Level 3: Triangles, Coverage and Depth

The same models are also stored as triangles, with 3 vertex indices and a color. Drawing does not use edges, instead:

1. First, project the three vertices.
2. Next, take a bounding box, a min/max of the projected points, then clipped to 320x200.
3. For each of the pixels in that box, then test the pixel center with the edge function, the signed area. If all of the 3 weights have the same sign, then the pixel is inside (barycentric).
4. Finally, interpolate z across the triangle and then compare it to a depth buffer. Only the nearer triangle keeps the pixel. Then the color buffer is drawn as 5x5 squares, same as level 2. The closer trees cover the objects behind them, the occlusion part as described in the project specifications.



Future Work

Level 3 works as I originally planned it to, but I could also add logic where it redraws when the camera actually moves, and I would skip the pixels faster to make it possibly look smoother. Also, since I had a lot of fun designing the objects, adding more variability in the objects could make the forest look more natural, maybe add some lighting or a ground mesh to make the interactive experience more immersive in a way. A possible next version could be adding ground patches, a small camera bob, and a lighter version of the win so that the text message is itself drawn into the pixel grid so the game looks more aesthetically like a game upon winning.

AI Usage

My AI usage for this project was typically used in assistance in development in helping me shape my ideas into proposed solutions, with me designing the logic and implementing a version of it myself, and then debugging it with AI, especially when logic and syntax errors required fixing. It also helped with randomly scattering the trees and rocks when one of those objects was made, finishing the cabin including helping design the collision logic so you cannot walk through the objects, and also debugging the pay-respects function with the win logic. It also did minor cleanup that helped finish the 3 levels.